Advanced Database Systems:

Part III.5 Column-Oriented Databases

Reinhard Pichler

Institute of Logic and Computation
"Databases & Artificial Intelligence" Group

Summer Semester 2023

TU WIEN Informatics

References

- Textbooks:
  - Pramod J. Sadalage and Martin Fowler:
    NoSQL Distilled. Addison-Wesley 2013
  - Lena Wiese:
    Advanced Data Management for SQL, NoSQL, Cloud and Distributed Databases, de Gruyter, 2015
- Further articles and books:
  - Daniel Abadi, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden: The Design and Implementation of Modern Column-Oriented Database Systems.
    Foundations and Trends in Databases, 5(3): 197-280, 2013
- Copyright of all figures is with one of these resources.

Contents

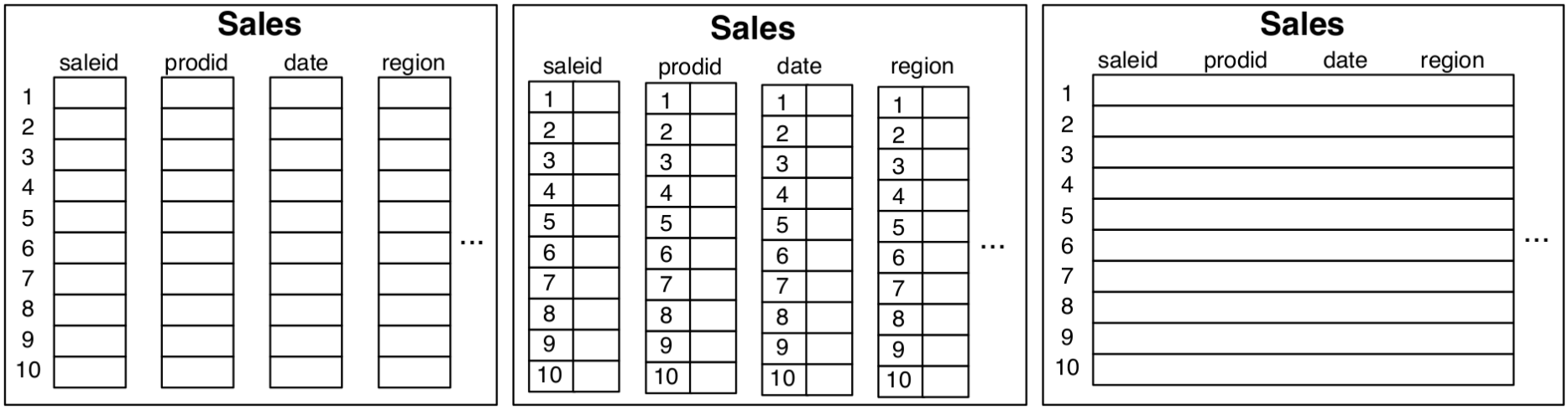
- Column Stores:
  - Basic Ideas
  - Explicit vs. Implicit IDs
  - Compression
  - Late Materialization
  - Updates
- Column Family Stores (Wide Column Stores):
  - Motivation
  - Data Model

Contents

- Column Stores:
  - **Basic Ideas**
  - Explicit vs. Implicit IDs
  - Compression
  - Late Materialization
  - Updates
- Column Family Stores (Wide Column Stores):
  - Motivation
  - Data Model

Column Stores

- Basic Idea:
  - Physically store data in vertical fragments
  - This is in contrast to traditional RDBMSs, which store whole records together (= row store)



(a) Column Store with Virtual Ids

(b) Column Store with Explicit Ids

(c) Row Store

Why Column Stores?

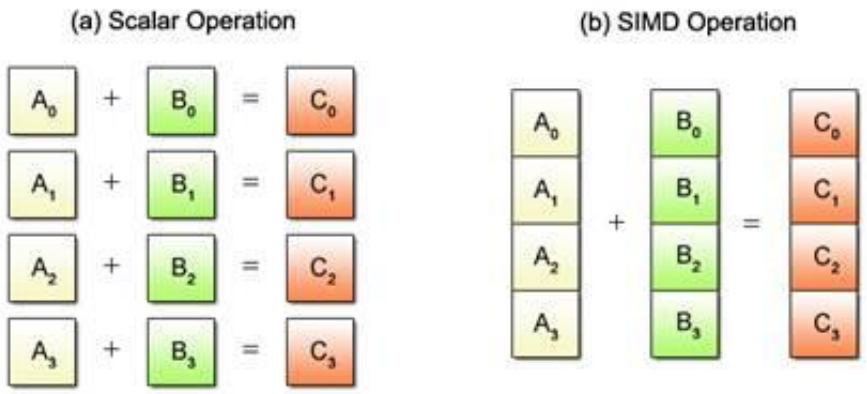
- Business Analytics & Data Warehouses:
  - For analytics you want to store as much information as possible, hence tables contain many columns.
  - Concrete query usually involves aggregation over few columns, e.g., product sales in a given month: involves only product id, date, maybe amount.
- With Column Stores:
  - Save I/O by reading only the columns you need;
  - further optimizations of column-wise processing.

Historical Perspective

- Many of the ideas are old:
  - Column-oriented layouts were proposed in the 1970s as part of research on time-series databases.
  - Why the recent (around 2000) surge in popularity?
- Development of server hardware since the 1980s:
  - CPUs have become ~10,000 times faster.
  - Cache/RAM/Disk size has also grown by a factor of ~10,000
  - Transfer from disk has become ~100 times faster.
  - Seek times have become ~10 times faster
- Consequences:
  - Trade-off between CPU and I/O has changed;
  - important for column-oriented optimizations.

Vectorized Processing

- Iterating over single columns instead of rows enables further speedups beyond the saved I/O.
- Modern CPUs support single instruction, multiple data (SIMD) operations: particularly well suited for column-wise processing
- Even without SIMD: Better cache locality in comparison to iterating over rows.



Source: https://kernel.org/pub/linux/kernel/people/geoff/cell/ps3-linux-docs/

Systems

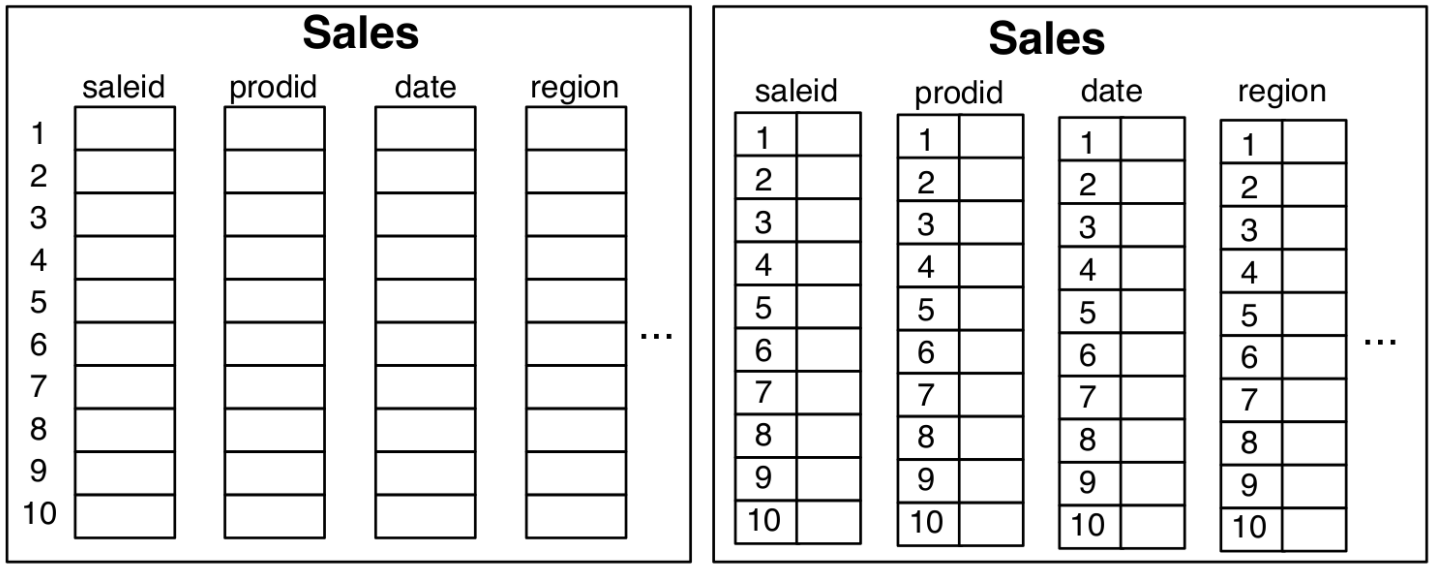
- Column-oriented extensions for other systems:
  - cstore_fdw for PostgreSQL
  - "columnstore indexes" in Microsoft SQL Server
  - in-memory column store of Oracle
  - SAP HANA stores data in column *and* row format
- "Pure" column stores:
  - Open source: MonetDB, Apache Kudu (on Hadoop), Druid (Apache)
  - Commercial: Vertica (commercial fork of earlier C-Store system), Sybase IQ (an SAP company)

Contents

- Column Stores:
  - Motivation
  - **Explicit vs. Implicit IDs**
  - Compression
  - Late Materialization
  - Updates
- Column Family Stores (Wide Column Stores):
  - Motivation
  - Data Model

Need for Ids

- The system needs to be able to reconstruct the rows.
- Example: Which sales were made on which date?
- This means: We need a way to identify which values in the columns belong to the same row. ⇒ Ids

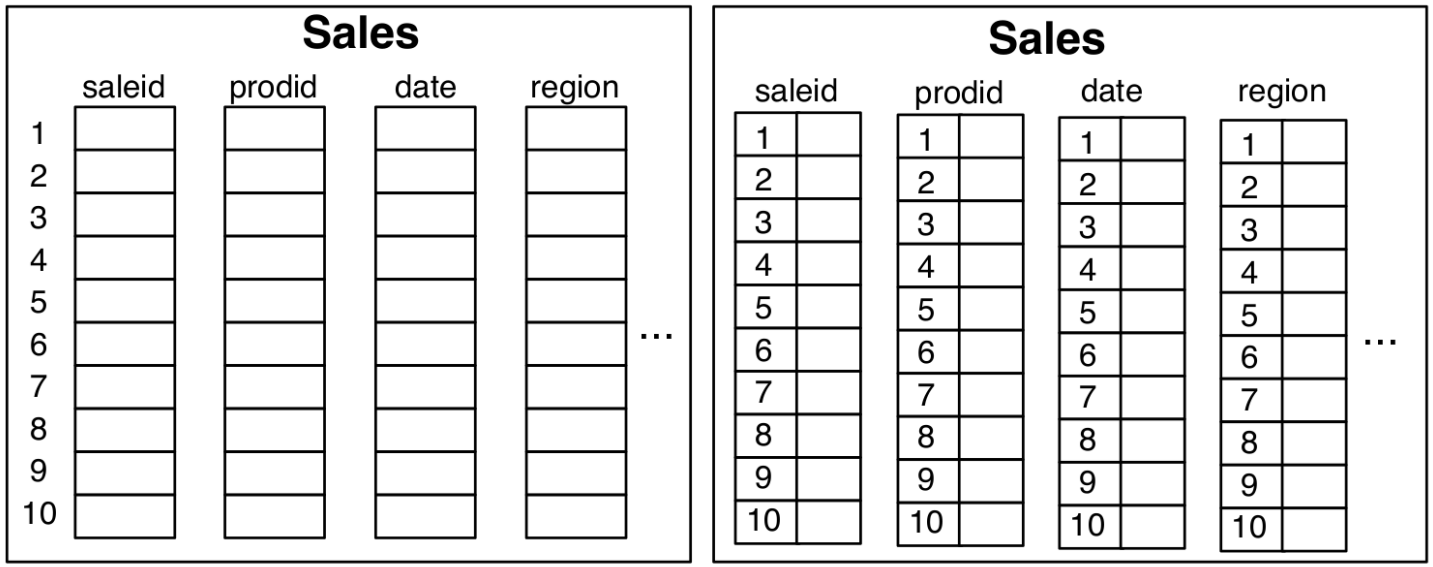


(a) Column Store with Virtual Ids

(b) Column Store with Explicit Ids

Explicit Ids

- Associate a tuple identifier with every entry in every column
- Disadvantage: data size increases ⇒ more I/O

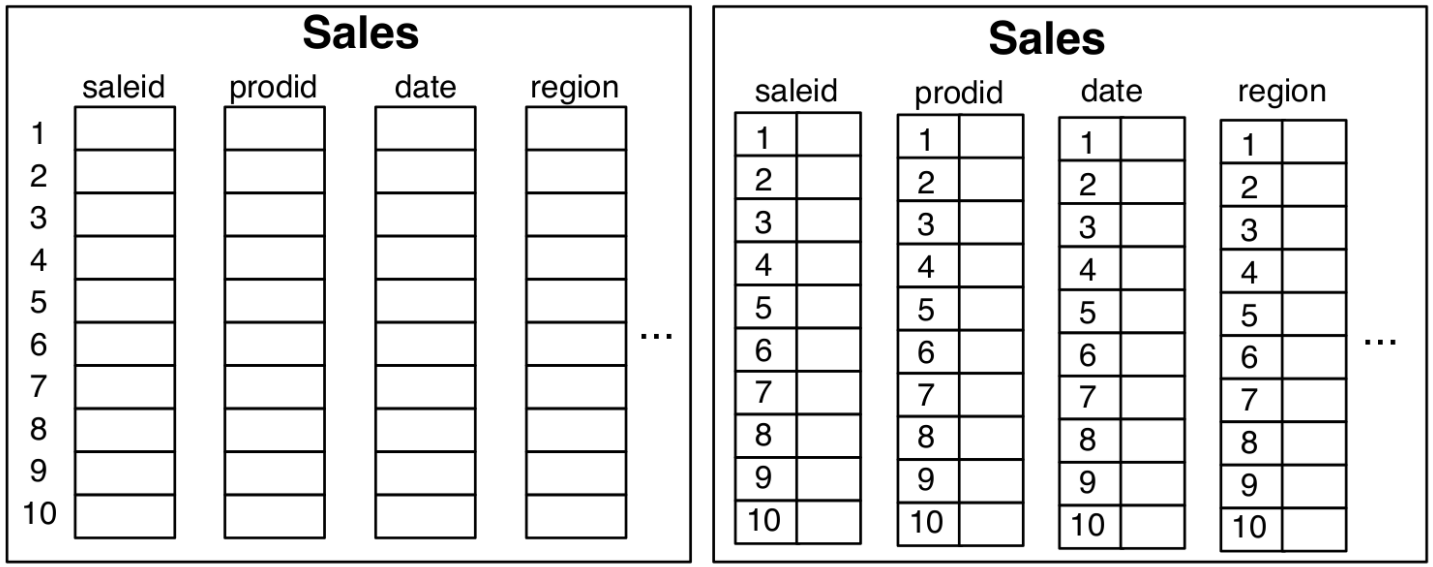


(a) Column Store with Virtual Ids

(b) Column Store with Explicit Ids

Virtual Ids

- Avoid Id-column by using the offset as virtual Id.
- Efficient lookup via offset: startOf(C) + i * width(C)
- Disadvantage: problem with (non-fixed length) compression methods and variable-width columns.



(a) Column Store with Virtual Ids

(b) Column Store with Explicit Ids

Contents

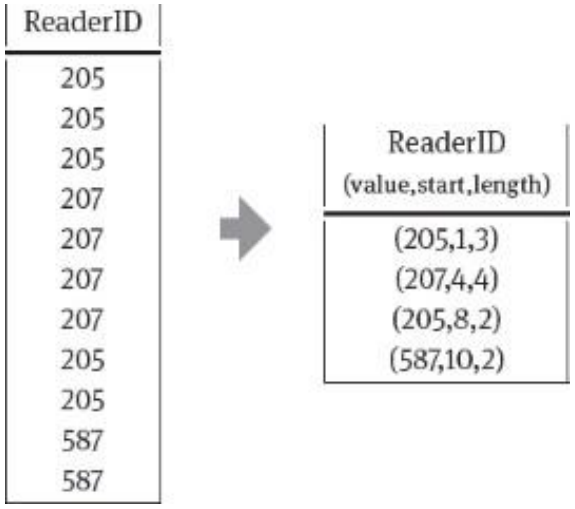
- Column Stores:
  - Motivation
  - Explicit vs. Implicit IDs
  - **Compression**
  - Late Materialization
  - Updates
- Column Family Stores (Wide Column Stores):
  - Motivation
  - Data Model

Compression – Motivation

- Analytical queries usually require a lot of I/O and aggregation (cheap for the CPU)
- Idea: trade CPU time for I/O by storing compressed data
  - Column-oriented layout allows for specialized choice of compression algorithm depending on e.g. data types.
  - Single columns have similar values (high data locality) ⇒ yet better compression possible.
  - Example: Compressing only product price vs. compressing product price + picture + description + ...
  - Drawback: Indexing and offset addressing may become more difficult.

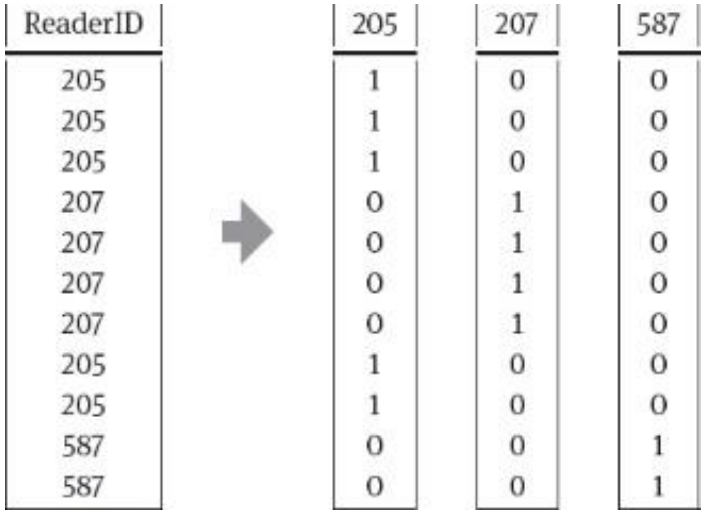
Run-Length Encoding

- Represent consecutive repetitions of values as pair (value, #repetitions).
- Worse than no compression in case of little repetition (or even unique values)
- Disadvantage: Causes issues with virtual ids.



Bit-Vector Encoding

- One bit-string with length of the number of entries in the column for each value.
- 1 in i-th position encodes value (cf. Bitmap Index)
- Only useful with few possible values.
  (Exception: further compression of the bit-strings.)

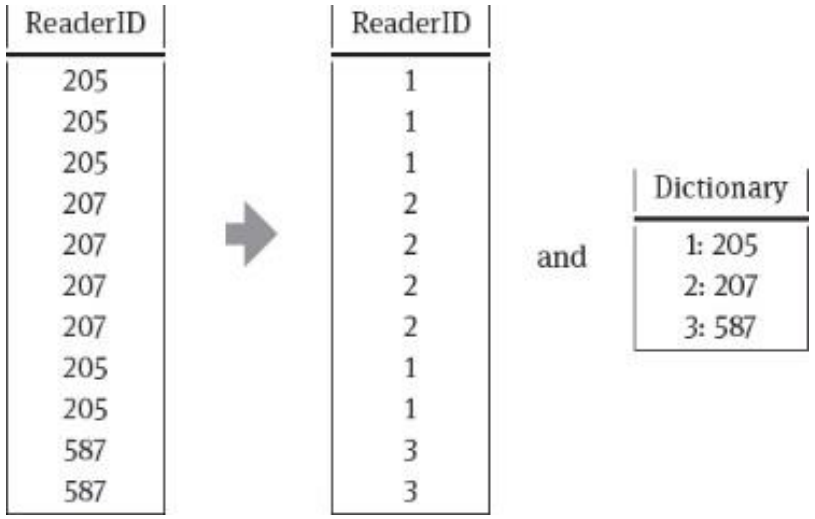


ReaderID
205
205
205
207
207
207
207
205
205
587
587

205	207	587
1	0	0
1	0	0
1	0	0
0	1	0
0	1	0
0	1	0
0	1	0
1	0	0
1	0	0
0	0	1
0	0	1

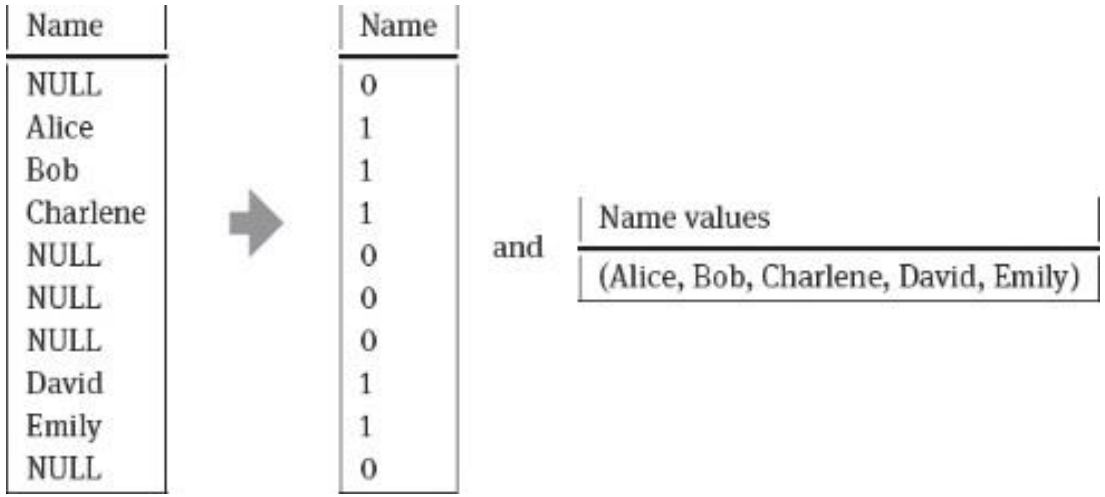
Dictionary Encoding

- Create dictionary from values and use integer indices into the dictionary instead of the actual values.
- Replaces long values with shorter placeholders (e.g., 4 byte integer instead of a name string)
- Can turn a variable-width column into fixed-width



Null Suppression

- Some applications lead to *sparse* columns, i.e., columns with many NULL values.
- Idea: Store sparse columns without the NULL values.
- Example: Position bit-string encoding below. Other null suppression techniques also exist.



Name
NULL
Alice
Bob
Charlene
NULL
NULL
NULL
David
Emily
NULL

Name
0
1
1
1
0
0
0
1
1
0

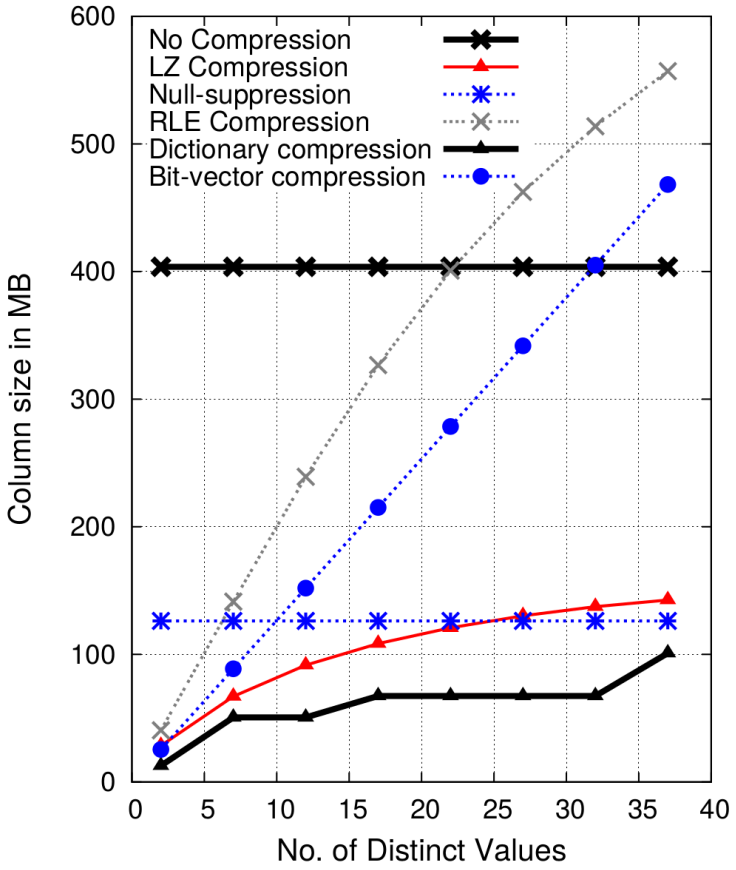
and

Name values
(Alice, Bob, Charlene, David, Emily)

Operations on Compressed Data

- More intricate compression schemes -- with better compression ratios, e.g. Lempel-Ziv, Huffman coding, etc.
- Ideal solution: Ultimate performance boost if we can operate directly on the compressed data! This saves I/O and eliminates the need for decompression.
- With the presented compression schemes, many common operations are in fact possible without decompression. Example: SUM over a run-length encoded column. Directly add <value> * #repetitions.
- In general, requires special consideration by the query execution engine. Results in highly complex systems.

Real World Performance (Space Reduction)



- Column with ~100 million 32bit integers.
- Significant improvements even with very simple methods
- Similar improvements observed in query execution time.

Source: Abadi, Madden, Ferreira. "Integrating compression and execution in column-oriented database systems."

Compression – Conclusion

- There are many compression strategies to consider.
- Many allow (some) operations on compressed data.
- Concrete choice of compression strategy is column- and application-specific.
- Example:
  - “date" column in table of invoices: long sequences of same date expected ⇒ run-length encoding
  - "month" of birth dates of users: no regularity expected, but only 12 values ⇒ bit-vector encoding
  - address column ⇒ dictionary

Contents

- Column Stores:
  - Motivation
  - Explicit vs. Implicit IDs
  - Compression
  - **Late Materialization**
  - Updates
- Column Family Stores (Wide Column Stores):
  - Motivation
  - Data Model

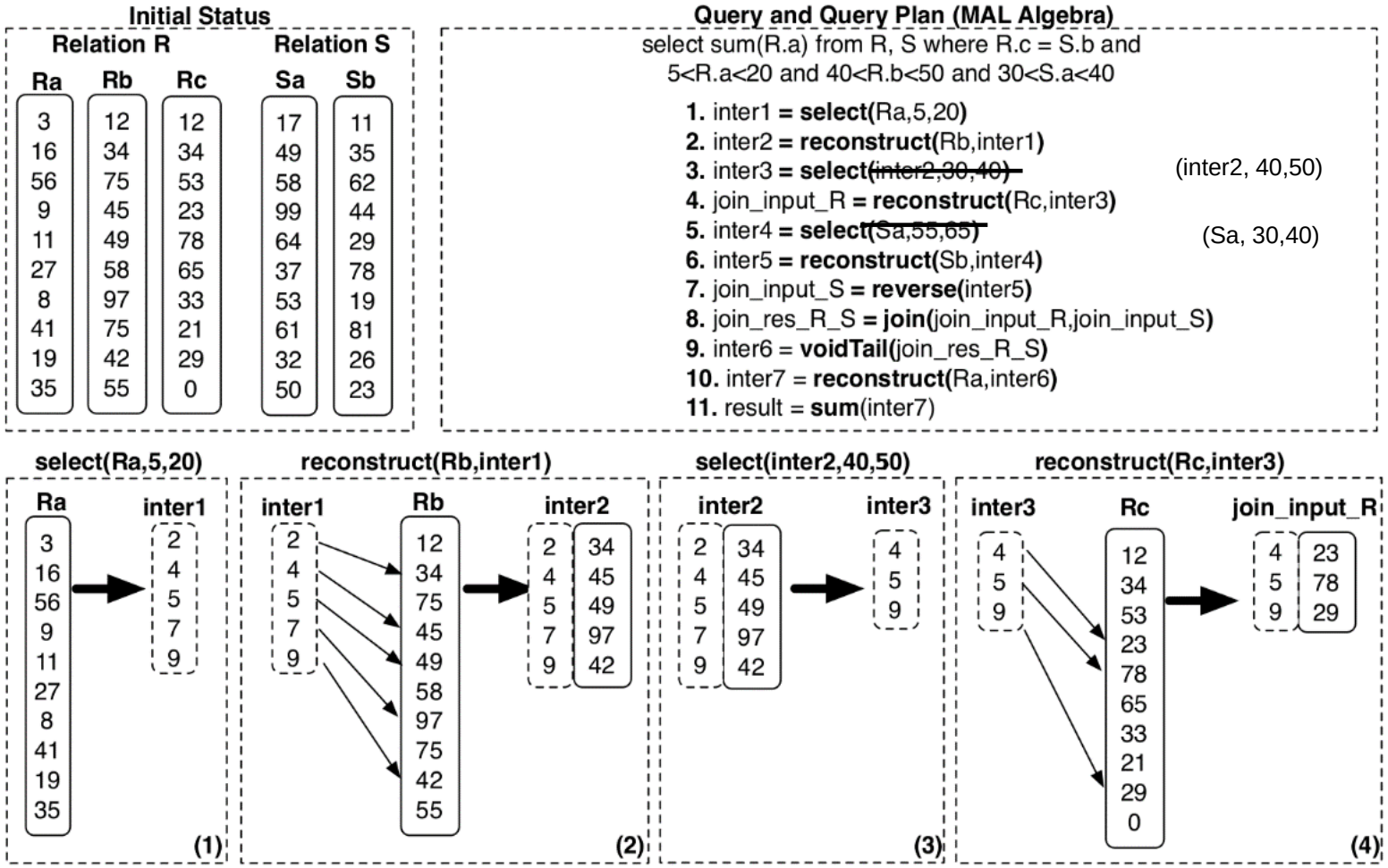
Materialization

- Queries often access multiple attributes of a table. Thus, the separately stored columns need to be brought together at some point. This is called materialization.
- Early materialization: Reconstruct (materialize) the rows right after reading the columns. Simple but we lose most of the column-oriented performance improvements:
  - Materialization may require decompression.
  - No or very limited vectorized processing.
- Better solution: late materialization, i.e., materialize as late as possible.

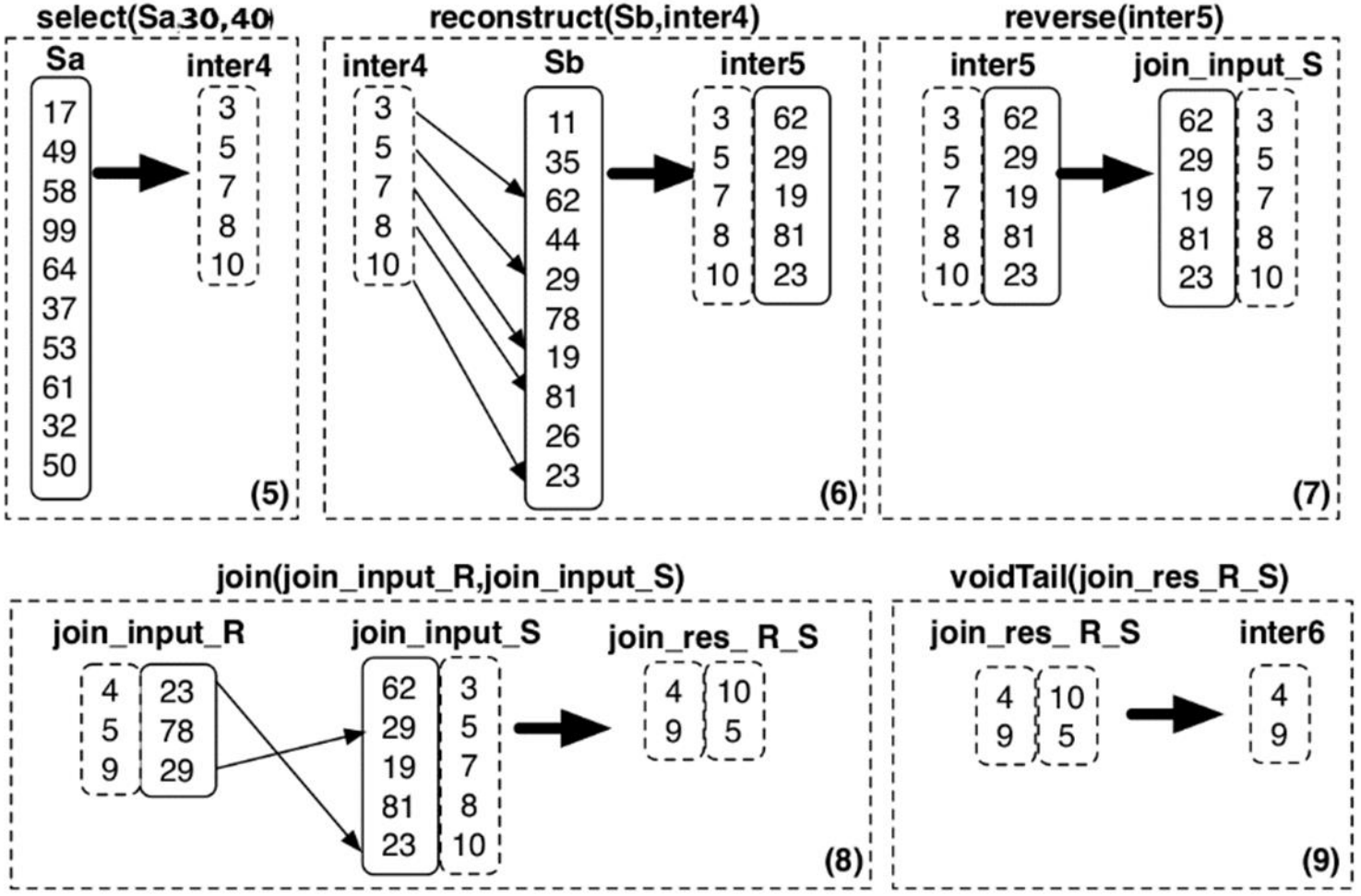
Late Materialization

- Operate on individual columns as much as possible to exploit column-oriented optimizations.
- Combine columns as late as possible.
- For some aggregation tasks, it may be possible to completely skip materialization.
- Requires keeping track of intermediate position tables

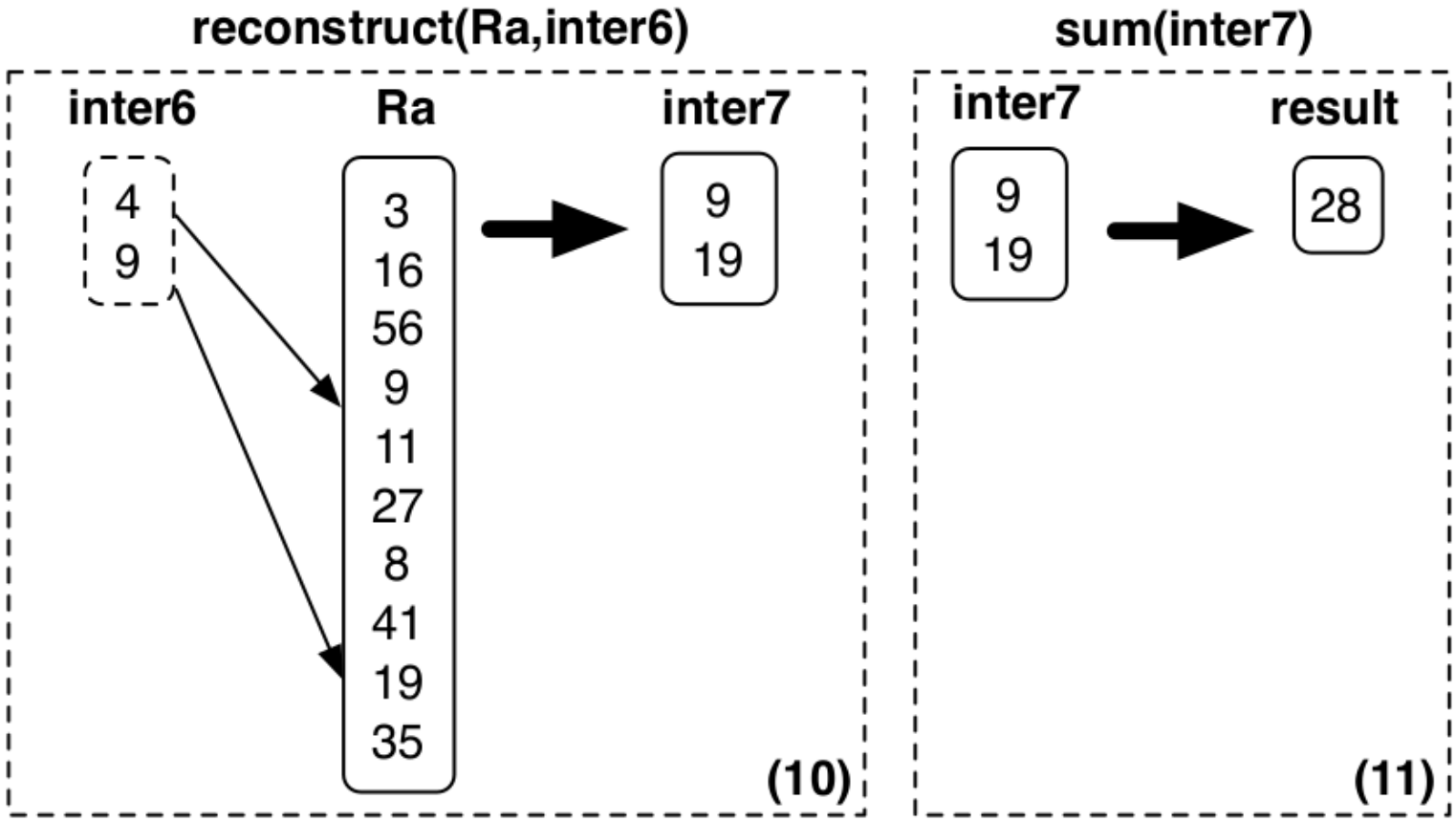
Late Materialization –Example



Late Materialization –Example (Cont.)



Late Materialization –Example (Cont.)



Contents

- Column Stores:
  - Motivation
  - Explicit vs. Implicit IDs
  - Compression
  - Late Materialization
  - **Updates**
- Column Family Stores (Wide Column Stores):
  - Motivation
  - Data Model

Updating Columns

- Storing columns in separate files means multiple writes (all columns) in case of insertion and deletion.
- Inserting and deleting in compressed columns can be very expensive and complex. In the worst case, this requires: decompress, update, recompress.
- In consequence, many systems manage a separate write-store for recent updates in addition to the read-store (= main database).
- Queries are executed on both the read- and the write-store and the results are merged on-the-fly.
- Contents of the write-store is periodically propagated to the read-store to keep the write-store small.

Contents

- Column Stores:
  - Motivation
  - Explicit vs. Implicit IDs
  - Compression
  - Late Materialization
  - Updates
- Column Family Stores (Wide Column Stores):
  - **Motivation**
  - Data Model

Pioneering Work at Google: Bigtable

Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Michael Burrows, Tushar Chandra, Andrew Fikes, Robert E. Gruber:

Bigtable: A Distributed Storage System for Structured Data. ACM Trans. Comput. Syst. 26(2): 4:1-4:26 (2008)

Also: Best paper at OSDI 2006:

(Symposium on Operating Systems Design and Implementation)

Pioneering Work at Google: Bigtable

- Motivation: huge amount of data
  - Web pages:
    contents, crawl metadata, links, anchors, page rank
  - Per-user data:
    user preference settings, recent queries/search results
  - Geographic locations:
    physical entities (shops, restaurants, etc.), roads, satellite image data, user annotations
- Principal challenge: scale
  - #web pages, many versions per page
  - #users, #queries per second
  - size of image data

Requirements

- Up-to-dateness:
  - Data has to be continuously updated by asynchronous processes.
  - Allow access to most current data at any time.
- Performance requirements:
  - Very high read/write rates (millions of ops per second)
  - Efficient scans over all the data or interesting subsets
- Temporal perspective:
  - Examine data changes over time,
    e.g.: contents of a web page over multiple crawls.

Characteristics of Bigtable

- Distributed, multi-level, sparse map:
  - Row key – column – timestamp
- Fault-tolerant, persistent
- Performance:
  - Fast access to very large amounts of data (structured, semi-structured, unstructured)
  - High read and write throughput
- Scalability:
  - Dynamically add / remove cluster nodes
  - Scale seamlessly from thousands to millions of reads/writes per second
  - Automatic replication

Building Blocks (from Google Infrastructure)

- Google File System (GFS): to store log and data files
- SSTable (Sorted Strings Table):
  - immutable file format to store log and data files
  - Key/value string pairs sorted by key
  - consists of sequence of blocks (64kB default) + block index (loaded to memory when SSTable is opened).
- Chubby:
  - highly available, persistent distributed lock service
  - 5 active replicas with 1 elected master
  - entry point to Bigtable data

Contents

- Column Stores:
  - Motivation
  - Explicit vs. Implicit IDs
  - Compression
  - Late Materialization
  - Updates
- Column Family Stores (Wide Column Stores):
  - Motivation
  - **Data Model**

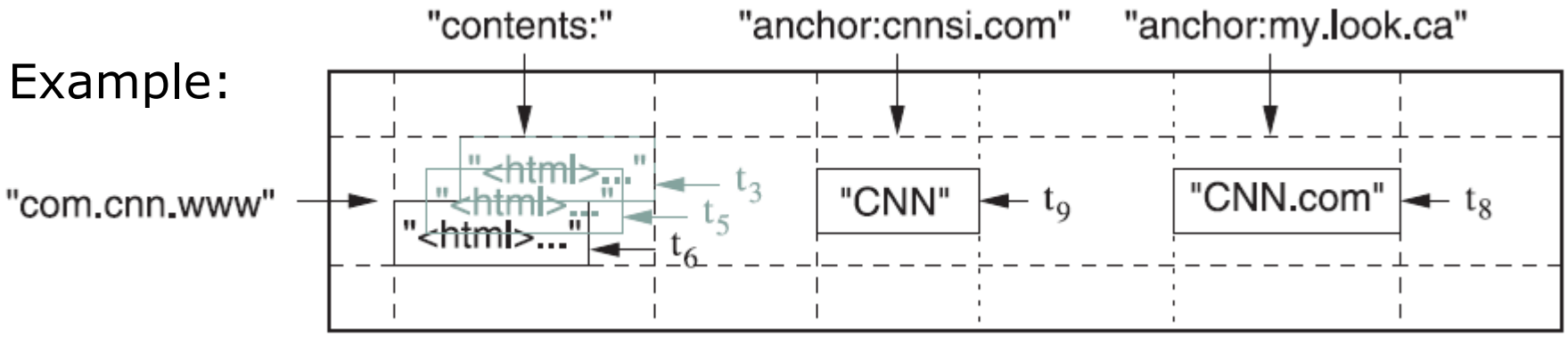
Rows

- Data is stored in lexicographic order by key
- Key can be arbitrary string (≤ 64kB, usually much less)
- Rows with consecutive keys are grouped into tablets (= the unit of distribution and load balancing).
  - "neighbouring" rows on 1 or small number of nodes
  - applications can choose key to achieve good locality (e.g., store web pages with reversed url as key).
- Each tablet consists of one or more SSTables.
- Access to data in a row is atomic.

Columns

- Columns are grouped into column families:
  - Two level name structure: *family:optional_qualifier*
  - Small number of column families recommended (100s)
  - Unbounded number of columns in a family
- Column family:
  - Unit of access control
  - Must be created explicitly before data can be stored.

Example:



Timestamps

- Cells in a table can contain multiple versions of data
  - Can be assigned implicitly by Bigtable ("real time" microseconds) or explicitly by the application.
  - Cell versions are stored in decreasing timestamp order.
- Lookup options:
  - "Return most recent K values" or
  - "Return all values in a given timestamp range" or all.
- Garbage collection options (per column family):
  - "Only retain most recent K values"
  - "Retain values until they are older than K seconds".

Data Access via API

- Schema:
  - Create/delete tables or column families
  - Change metadata (such as access rights)
- Read access:
  - Single-row reads (based on row key)
  - Scans (range of rows, multiple ranges of rows)
- Write access:
  - Simple write, batch write (ideally contiguous rows)
  - Modifications (e.g., increment integers, append data)
- Read/write data in Bigtable with MapReduce

Lessons Learned (according to Google)

- Prepare for many types of failure:
  - not just single node failure or network partition
  - also e.g. memory/network corruption, huge clock skew, hung machines, bugs in other systems, etc.
- Delay adding new features until their usage is clear:
  - Originally, general purpose transactions were planned;
  - ultimately, mostly single-row transactions were needed.
- Importance of proper system-level monitoring
  - Bigtable itself, but also client processes using Bigtable
- "Most important" lesson: simple design

Further Popular Systems

- HBase: Bigtable-like open source implementation
  - PC/EC according to PACELC categorization
  - master-based
  - access only via APIs (no in-built query language)
  - built on top of Hadoop and HDFS:
    good integration with Hive, Spark, etc.
- Cassandra: quite different from Bigtable
  - PA/EL according to PACELC categorization
  - masterless, data model different from Bigtable
  - Cassandra Query Language (CQL): SQL-like

Summary

- Column stores: basic ideas, motivation
- Basic principles of column stores:
  - Explicit vs. implicit ids
  - Compression: motivation, methods
  - Materialization: early vs. late materialization
  - Updating columns: challenges, solutions
- Wide column stores (column family stores):
  - Motivation, requirements, characteristics
  - Data model and data access
  - Common systems: Bigtable, Cassandra, HBase